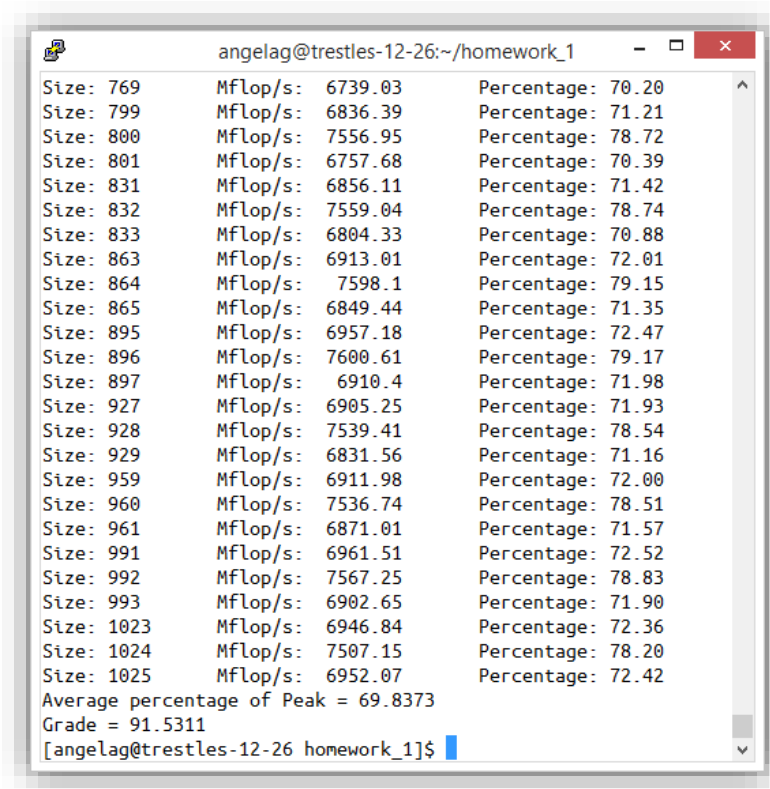


SQUARE DGEMM OPTIMIZATION

ANGELA GROSS // CSCI-595 // HOMEWORK 1

OVERVIEW

All in all, this homework was extraordinarily difficult, but it was a good experience and I'm happy to have had the experience of working intimately with C. Starting with reading through PHiPAC and the other papers that the other students presented and ending with fine-tuning my compiler commands, I finally achieved a final efficiency of 69.8%:



The screenshot shows a terminal window titled 'angelag@trestles-12-26:~/homework_1'. It displays a table of performance metrics for various matrix sizes. The table has three columns: 'Size', 'Mflop/s', and 'Percentage'. The data is as follows:

Size	Mflop/s	Percentage
769	6739.03	70.20
799	6836.39	71.21
800	7556.95	78.72
801	6757.68	70.39
831	6856.11	71.42
832	7559.04	78.74
833	6804.33	70.88
863	6913.01	72.01
864	7598.1	79.15
865	6849.44	71.35
895	6957.18	72.47
896	7600.61	79.17
897	6910.4	71.98
927	6905.25	71.93
928	7539.41	78.54
929	6831.56	71.16
959	6911.98	72.00
960	7536.74	78.51
961	6871.01	71.57
991	6961.51	72.52
992	7567.25	78.83
993	6902.65	71.90
1023	6946.84	72.36
1024	7507.15	78.20
1025	6952.07	72.42

Below the table, the following summary statistics are shown:

- Average percentage of Peak = 69.8373
- Grade = 91.5311

The terminal prompt is '[angelag@trestles-12-26 homework_1]\$'.

METHODS, APPROACHES, AND DIFFICULTIES

I tried a couple of different methods, (as apparent from the 36 versions of my C file, dgemm-optimized.c), but I'll only talk about the methods that I spent the most time on.

UNROLLED, 4X4 SSE2 DGEMM

By Robert's suggestion, I was able to find and easily follow a tutorial on a wiki, <http://wiki.cs.utexas.edu/rvdg/HowToOptimizeGemm>, and I was able to implement an unrolled-by-4 matrix multiply with SSE2. This gave a huge boost in performance, which was from about 10% (naïve) to 40%.

However, one of the parts that I spent the majority of my time on was the related edge-cases to the 4x4 dgemm, which was when the dimensions of the square matrices weren't divisible by 4. There were three ways I could have handled this: a) Copy over the arrays A, B, and C to a padded-zero array that was divisible by 4, b) Handle the fringes naively, or c) Try to cleverly divide the fringe cases to 2x2, 3x3, and 1x1 (naïve) blocks. I decided to try (a) and (b), but I had the most success with (a). As I haven't done a ton of C programming, I struggled with coming up with a clever and efficient way to

copy over the values to a padded array without complicated pointer arithmetic. After about 10-15 hours of drawing diagrams, testing basic C code online, and becoming familiar with GDB so I could debug the program easily, I finally found a way to copy over the values in $O(n)$ time. As a result, I'm very proud of that padding code!

I also tried to make an 8x8 version, but this lowered the efficiency to 30% (even after blocking and with dimensions that are multiples of 8). I'm guessing this had to do with the overhead involved with dealing with such a large block of memory.

BLOCKED DGEMM

Again, I used the tutorial from the wiki (see above), but I struggled with a bug that would result in wrong answers if $n > \text{block_size}$. After spending over 5 hours debugging this, I found that I was simply overwriting the value(s) of C in my vectorization code instead of adding it on (i.e. I did $C_{12} := c_{12}$ instead of $C_{12} := C_{12} + c_{12}$). What a frustrating bug!

SSE3 DGEMM

After realizing that I could use SSE3 methods on Trestles, I replaced one of my SSE2 methods with an SSE3 method, and it boosted my efficiency from 54% to 68%!

CONCLUSION

Although this homework was difficult, gaining the knowledge of efficient practices in C was invaluable. The tools/methods needed in order to multiply matrices optimally is applicable to nearly any program written in C, and so it felt like an assignment that I got a lot out of.